TCSS 343 – Design and Analysis of Algorithms

Deterministic Polynomial Time

Complexity Theory

- In computational complexity theory we attempt to classify problems based on how difficult (complex) they are.
  - Computational_complexity_theory
  - Generally more complex problems require more resources to solve.
  - Time and space are the commonly measured resources.
  - We also consider the type of computing done:
    - Deterministic
    - Probabilistic
    - Quantum
    - Non-Deterministic
- We classify problems based on the resources used by the best known solutions.
  - It is often very difficult or impossible to show that a problem has no better solution than the ones we know about.
    - Subset Sum
    - Partition
    - Knapsack
  - For some problems we long thought they did not have a better solution and we were wrong.
    - Primes

Decision Problems

- Since problems all have a different input and output structure it is not easy to compare them.

Search an Unsorted List (SAUL)
Input: $A[1 \dots n]$ is a list of integers and t is an integer
Output: $i \in \{1, \dots, n\}$ such that $A[i] = t$ if such exists
-1 otherwise.

Integer Multiplication (MULT)
Input: Integers N and M
Output: NM an integer equal to the product of N and M

- Every problem will have a different input pattern and this is somewhat unavoidable.
  - We handle this by assuming that every input is encoded in binary or another formal alphabet as a string.
- We can force every problem to have the same output structure by only considering decision problems.
- A decision problem will always answer either “yes” or “no” / “true” or “false” / “1” or “0”.
  - Decision_problem

- Can we rewrite SAUL and MULT as decision problems?

Decision Problems

- We can convert most problems into a decision problem by adding an additional input.

Search an Unsorted List (SAUL)
Input: $A[1 \dots n]$ is a list of integers and t is an integer and $i \in \{-1,1, \dots, n\}$
Output: "yes" if $A[i] = t$ or if $i = -1$ and $A[i] \neq t$ for all $1 \leq i \leq n$
"no" otherwise.

Integer Multiplication (MULT)
Input: Integers N and M and P
Output: "yes" if $NM = P$
"no" otherwise.

- Once we are considering a decision problem we can see the problem as dividing the input into those inputs that are accepted and those that are rejected.

- As a result each decision problem is equated with a set of inputs.
  - Remember that inputs are all encoded as binary strings so each set is a set of binary strings.
  - Each problem (at least the non-trivial ones) has infinite binary strings in it.

- Computational complexity theory creates classes of problems that are formally interpreted as classes of sets of binary strings.

Time Complexity

- Consider classifying problems based on their time complexity (the amount of time used by their best solutions).

- We have already been classifying run times in classes using asymptotic notation. For example:
  - $\Theta(\log n)$
  - $\Theta(n)$
  - $\Theta(n \log n)$
  - $\Theta(n^2)$

- One problem with this method is it creates far to many classes.
  - There are infinite classes $\Theta(n^i)$ for every real number i greater than one.
  - There are infinite classes $\Theta(\log^i n)$ for every real number i greater than one.
  - Then we can get infinite more by multiplying these functions. $\Theta(n^i \log^i n)$

- In complexity theory we are mostly interested in those problems that are efficiently solvable.
  - Our threshold for efficiency is polynomial time.
  - P is the class of decision problems that can be solved in deterministic polynomial time.

Polynomial Time

- The complexity class P is the class of all decision problems that can be solved by a deterministic algorithm in polynomial time.
  - P (complexity)

- Polynomial time is defined as the union of all classes $O(n^i)$ for all $i \geq 0$.

- How can we show a problem p is in P?
  - We develop a deterministic algorithm A.
  - We show A is correct for p.
  - We show that A runs in $O(n^i)$ time in the worst case.

- Example: Search an Unsorted List (SEARCH)
  - Consider the deterministic algorithm sequential search.
  - Sequential search is easy to prove correct.
  - Sequential search runs in time $O(n)$.
  - So SEARCH is in P.

- Example: Integer Multiplication (MULT)
  - Consider the deterministic grade school algorithm.
  - The grade school algorithm is correct.
  - The grade school algorithm requires $O(n^2)$ operations.
  - So MULT is in P.

Case Study: Primes

- Consider the problem of identifying primes.
  - Primality test

```
Primes
Input:    N an integer greater than 1
Output:   "yes" if N is prime
          "no" otherwise
```

- What is a simple algorithm for this problem?
  - Brute force: divide N by every integer from 2 to $N-1$.
  - This algorithms takes $O(N)$ operations.
  - Can we improve this algorithm?
    - We can stop checking at $\sqrt{N}$. This algorithm takes $O(\sqrt{N})$ operations.
    - We only need to divide N by primes in the range.

- Does this algorithm show that Primes is in P?
  - No. We are overlooking that the actual size of the input N is $\lfloor \log N \rfloor + 1$ bits.
  - Thus, these algorithms take exponential time in the size of N.

- While these algorithms do not show that Primes is in P many researchers thought that there must be an algorithm for Primes that runs in polynomial time while others thought this was impossible.

Case Study: Primes

- Consider the problem of identifying primes.
  - Primality_test

```
Primes
Input:    p an integer greater than 1
Output:   "yes" if p is prime
          "no" otherwise
```

- In 1976 Gary Miller gave an algorithm that appeared to run in polynomial time.
  - He could not prove that it did without relying on an unproven mathematical conjecture (the Riemann Hypothesis) about the distribution of primes.
    - Generalized_Riemann_hypothesis
  - In 1980 Rabin turned Gary's algorithm into a probabilistic algorithm that was easier to prove polynomial time.
    - Miller-Rabin_primality_test

- However membership in the class P requires a proof that a deterministic algorithm runs in polynomial time.
  - The Miller-Rabin algorithm was sufficient to show that Primes was in a related probabilistic class called BPP but not P.
  - Other randomized algorithms were also developed like the Solovay-Strassen_primality_test.

- In 2002 a deterministic algorithm that ran in polynomial time was developed by three Indian students.
  - Manindra Agrawal, Neeraj Kaya, and Nitin Saxena
  - AKS_primality_test
  - Finally there was a proof that Primes is in P!

Outside P

- The Primes problem demonstrates an important lesson.
  - We can never be sure that our best algorithm at the moment is really the best algorithm possible.
  - For any problem that we think is not in P there may one day be an algorithm to show the problem is in P.

- However we can define other complexity classes other than P and we might be able to show our problems are in these other classes.
  - We saw while looking at the Primes problem another class called BPP which stands for *bounded probability polynomial time*.
  - BPP is the class of problems that can be solved with a probabilistic algorithm in polynomial time with error at most $\frac{1}{3}$.
  - BQP is the class of problems that can be solved with a quantum algorithm in polynomial time with error at most $\frac{1}{3}$.
  - The problems in BPP and BQP are also considered efficiently solved if you are alright with an chance of error (and if you have a quantum computer).
  - We can also consider computation that doesn't (or could ever) exist. One such technology is called non-determinism.
  - NP is the class of problems that can be solved with a non-deterministic algorithm in polynomial time.

TCSS 343 – Design and Analysis of Algorithms

Non-Deterministic Polynomial Time

Non-Determinism

- Imagine you have a friend, Merlin, who claims to be a wizard that can compute the solution to any problem in a single second of thinking.

- You don't trust Merlin (since wizards aren't real and are tricksy) so you decide to come up with some tests.
- You will test Merlin by asking Merlin if a number is composite or not. You decide to pick some large numbers that would be hard to compute in your head quickly.
  - 5915587277
  - 5915587279
  - 5915587281
- You remember that exactly one of these three is prime but you don't want to try to compute this your self.

- You decide to ask Merlin for proof that a number is composite (provide a factor) or you will not trust him if he tells you the number is composite. He can only prove he is a wizard if he can convince you that two of the three numbers are composite.

- Merlin says that they are all composite (what a trickster)! You ask for proofs and he claims that the first number is divisible by 3, the second is divisible by 7 and the third is divisible by 3.
  - 5915587277 is not divisible by 3. So you catch Merlin in his lie.
  - $5915587279 = 7 \cdot 7 \cdot 811 \cdot 148861$
  - $5915587281 = 3 \cdot 19 \cdot 23 \cdot 359 \cdot 12569$

- Even though he lied he did provide correct proofs that the second two numbers are composite. Later you double check and find that the first number was indeed prime.

Verification Procedures

- The power of non-determinism is like having access to a wizard like Merlin. The wizard can compute anything but you can't trust him – he will try to mislead you if you are not careful.

- In non-deterministic computation you develop verification procedures to ensure that the proof given by the wizard is not a lie.

```
Composites
Input:   c an integer greater than 1
Output: "yes" if c is composite
         "no" otherwise
```

- This problem is the complement of the Primes problem.

- When we design a verification procedure for this problem, we always take an additional input which is the proof provided by Merlin. We also call this proof a witness or certificate.

```
CompositeVerifier(c, w)
         If w is not an integer return "no".
         If w ≥ c return "no".
         If w < 2 return "no".
         If c/w is not an integer then return "no".
         Return "yes".
End CompositeVerifier
```

Correctness of Verification Procedures

```
CompositeVerifier(c, w)
        If w is not an integer return "no".
        If w ≥ c return "no".
        If w < 2 return "no".
        If c/w is not an integer then return "no".
        Return "yes".
End CompositeVerifier
```

- A verification procedure is correct if:
  1. If the input should be accepted then there is some proof w such that the procedure will say "yes".
  2. If the input should be rejected then there is no proof w such that the procedure will say "yes".

- To prove a verification procedure correct we must prove an if and only if statement.
  - Our verifier returns "yes" if and only if we should say "yes" on the input. ($P \leftrightarrow Q$)

- This can be split into two conditional statements:
  - If we should say "yes" then we do say "yes". ($P \rightarrow Q$) (i)
  - If we do say "yes" then we should say "yes". ($P \leftarrow Q$) (ii)
- Recall that the contrapositive of a conditional is logically equivalent to the conditional so we two alternatives:
  - If we should do say "no" then we should say "no". ($\neg Q \rightarrow \neg P$) (i)
  - If we should say "no" then we do say "no". ($\neg Q \leftarrow \neg P$) (ii)
- Thus, it is sufficient to prove one of statement (i) and one of statement (ii).

Correctness of Verification Procedures

```
CompositeVerifier(c, w)
        If w is not an integer return "no".
        If w ≥ c return "no".
        If w < 2 return "no".
        If c/w is not an integer then return "no".
        Return "yes".
End CompositeVerifier
```

Lemma: If we should say "yes" then we do say "yes".

Proof: Assume we should say "yes" so c has a divisor between 2 and $c-1$. Assume w is this divisor. Then as an integer it passes test 1, and since it is between 2 and $c-1$ it passes test 2 and 3, and since it is a divisor it passes the final test and we say "yes".

□

Lemma: If we should say "no" then we do say "no".

Proof: Assume we should say "no" so c has no divisors between 2 and $c-1$. Assume w is an integer in the range 2 to $c-1$ so it has passed the first three tests. Since there are no divisors in this range we know w fails the last test and we say "no".

□

Polynomial Time Verification Procedures

```
Composites
Input:    c an integer greater than 1
Output:   "yes" if c is composite
          "no" otherwise

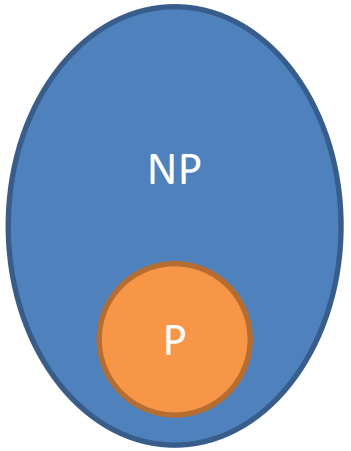
CompositeVerifier(c, w)
          If w is not an integer return "no".
          If w ≥ c return "no".
          If w < 2 return "no".
          If c/w is not an integer then return "no".
          Return "yes".
End CompositeVerifier
```

- We can also argue that this verification procedure is polynomial time.
  - The integer division can be carried out in $O(n^2)$ time where n is the number of bits in the integer.
- The class NP is the class of all decision problems that have polynomial time verification procedures.
  - NP_(complexity)
- So we have just shown that Composite is in NP.

P and NP

- It is easy to show that $P \subseteq NP$.
  - If a problem $p \in P$ then it can be solved by a deterministic algorithm in polynomial time.
  - We can create a polynomial time verification procedure that ignores the proof provided by Merlin and solves the problem instead.
  - This algorithm will only say "yes" on "yes" instances and "no" on "no" instances because it is correct for p by assumption.
  - It also runs in polynomial time by assumption.
  - So $p \in NP$ and equivalently $P \subseteq NP$.

- This results in two possibilities:
  - $P = NP$
    - The two classes are the same.
  - $P \subset NP$
    - The two classes are different.



- This problem remains open in computer science.
  - P_versus_NP_problem
  - You can earn a million dollars if you can solve it (plus a Ph.D.).
  - These days we are pretty sure that the $P \subset NP$ but no proof has been offered yet!

NP

- To show a problem p is in NP you must:
  - Design a verification V for p.
  - Prove V is correct.
  - Prove V runs in polynomial time.

- Let's try the problems:

Subset Sum(SUM)
Input: $S[1 \dots n]$ a list of n positive integers and t a target integer
Output: "yes" if there exists a subset of S that sums to t
"no" otherwise

Partition(PART)
Input: $S[1 \dots n]$ a list of n positive integers
Output: "yes" if the set S can be partitioned into two sets such that each has the same sum
"no" otherwise

Knapsack(0-1 KNAP)
Input: $W[1 \dots n]$ a list of n positive weights $V[1 \dots n]$ a list of n positive values, c the capacity of the knapsack and t a threshold value
Output: "yes" if there exists $A \subseteq \{1, \dots, n\}$ a set of indexes of items such that $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$
"no" otherwise

Subset Sum

```
Subset Sum(SUM)
Input:    S[1 … n] a list of n positive integers and t a target integer
Output:   "yes" if there exists a subset of S that sums to t
          "no" otherwise

SubsetSumVerifier(S[1 … n], t, W)
          If W ⊈ {1, … , n} return "no".
          If ∑_{i∈W} S[i] ≠ t return "no".
          Return "yes".
End SubsetSumVerifier
```

Lemma: If we should say "yes" then we do say "yes".

Proof: Assume we should say "yes" so S has a subset that sums to t. Assume $W \subseteq \{1, \dots, n\}$ selects this subset. Then as $W \subseteq \{1, \dots, n\}$ it passes the first test and since this subset sums to t it passes the final test and we say "yes". □

Lemma: If we should say "no" then we do say "no".

Proof: Assume we should say "no" so S has no subset that sums to t. Assume $W \subseteq \{1, \dots, n\}$ so it has passed the first test. Since S has no subset that sums to t this subset must not sum to t and W fails the last test and we say "no". □

Knapsack

Knapsack(0-1 KNAP)
Input: $W[1 \dots n]$ a list of n positive weights $V[1 \dots n]$ a list of n positive values,
c the capacity of the knapsack and t a threshold value
Output: "yes" if there exists $A \subseteq \{1, \dots, n\}$ a set of indexes of items such that $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$
"no" otherwise

Knapsack Verifier$(W[1 \dots n], V[1 \dots n], c, t, A)$
If $A \not\subseteq \{1, \dots, n\}$ return "no".
If $\sum_{i \in A} W[i] > c$ return "no".
If $\sum_{i \in A} V[i] > t$ return "no".
Return "yes".
End Knapsack Verifier

Lemma: If we should say "yes" then we do say "yes".

Proof: Assume we should say "yes" so there exists $A \subseteq \{1, \dots, n\}$ such that $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$. Assume $A \subseteq \{1, \dots, n\}$ is this set of items. So as $A \subseteq \{1, \dots, n\}$ it passes the first test and since $c \geq \sum_{i \in A} W[i]$ it passes the second test and since $\sum_{i \in A} V[i] > t$ it passes the final test and we say "yes".
□

Lemma: If we should say "no" then we do say "no".

Proof: Assume we should say "no" so there exists no $A \subseteq \{1, \dots, n\}$ such that $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$. Assume $A \subseteq \{1, \dots, n\}$ so it has passed the first test and that $c \geq \sum_{i \in A} W[i]$ so it passes the second test. Since no $A \subseteq \{1, \dots, n\}$ is such that both $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$ it must be the case that $\sum_{i \in A} V[i] > t$ and we say "no".
□

NP

- Let's try the problems:

Clique
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$.
Output: "yes" if there exists a complete subgraph of G of size k.
"no" otherwise

Independent Set
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$.
Output: "yes" if there exists a subgraph of G of size k with no edges.
"no" otherwise

Hamiltonian Cycle
Input: $G = (V, E)$ a connected undirected graph
Output: "yes" if there exists a simple cycle in G that visits every vertex exactly once
"no" otherwise

Clique

```
Clique
Input:    G = (V, E) an undirected graph and an integer k > 1.
Output:   "yes" if there exists a complete subgraph of G of size k.
          "no" otherwise

CliqueVerifier(G = (V, E), k, W)
          If W ⊈ V return "no".
          If |W| ≠ k return "no".
          If ∃_{u,v∈W}: (u, v) ∉ E return "no".
          Return "yes".
End CliqueVerifier
```

Lemma: If we should say "yes" then we do say "yes".

Proof: Assume we should say "yes" so G has a complete subgraph of size k. Assume $W \subseteq V$ is this complete subgraph. Then as $W \subseteq V$ it passes the first test and since $|W| = k$ is passes the second test. Finally since W is a complete subgraph it passes the final test and we say "yes".

□

Lemma: If we should say "no" then we do say "no".

Proof: Assume we should say "no" so G has no complete subgraph of size k. Assume $W \subseteq V$ and $|W| = k$ so it has passed the first two tests. Since G has no complete subgraph of size k this subgraph must not be complete and we fail the final test and we say "no".

□

Hamiltonian Cycle

```
Hamiltonian Cycle
Input:    G = (V,E) an undirected graph
Output:   "yes" if there exists a simple cycle in G that visits every vertex exactly once
          "no" otherwise

HCVerifier(G = (V,E), k, W)
          If W is not a permutation of V return "no".
          If ∃_{1≤i<n}: (w_i, w_{i+1}) ∉ E return "no".
          If (w_1, w_n) ∉ E return "no".
          Return "yes".
End HCVerifier
```

Lemma: If we should say "yes" then we do say "yes".

Proof: Assume we should say "yes" so G has a simple cycle that visits every vertex. Assume W is a permutation of V that visits the vertices in the order of this simple cycle. Then as W is a permutation of V it passes the first test and since it visits the vertices in the order of this simple cycle is passes the second test. Finally since $(w_1, w_n) \in E$ it passes the final test and we say "yes". □

Lemma: If we should say "no" then we do say "no".

Proof: Assume we should say "no" so G has no simple cycle that visits every vertex. Assume W is a permutation of V so it has passed the first test. Since G no simple cycle that visits every vertex this ordering must be missing an edge and we fail one of the final two tests and we say "no".

□

TCSS 343 – Design and Analysis of Algorithms

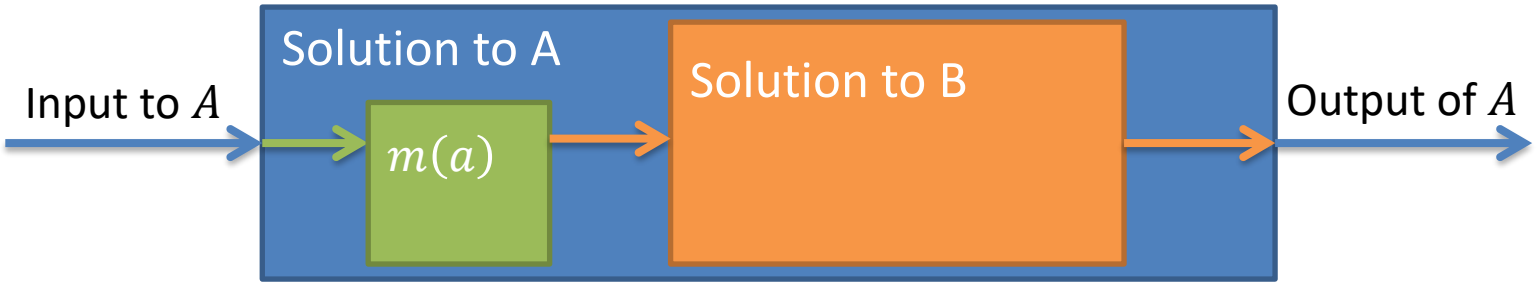
NP-complete

Reduction

- We have earlier introduced the concept of reduction as a relationship between problems.
  - Previously we stated that a reduction is a mapping from inputs to one problem to inputs to another problem that preserves the output in someway.
  - We applied this concept to algorithm design by finding self-reductions, or reductions from a problem to its sub-problems.
- We now have the tools to provide a more precise definition for another type of reduction.
  - Let A and B be decision problems.
  - We say that A reduces to B in polynomial time, denoted $A \leq_P B$, if there exists a mapping $m(a)$ that maps an input to A to an input to B such that:
    1. If a is a "yes" instance of A then $m(a)$ is a "yes" instance of B.
    2. If a is a "no" instance of A then $m(a)$ is a "no" instance of B.
    3. The mapping $m(a)$ can be computed in polynomial time.
- This type of reduction is called a polynomial time reduction (or a Cook reduction).

Reduction

- If we can show that $A \leq_P B$ then we have shown that A is *no harder* than B (up to a polynomial time amount of extra work).
  - Formally, if $A \leq_P B$ then we can take any solution for B and use it to solve A.
    - On input a compute $m(a)$.
    - Use the solution for B to compute the output of B on input $m(a)$.
    - Answer as B did.
  - Since m is polynomial time computable the first step will take polynomial time.
  - The second step will take as much time as the solution for B.
  - The last step takes constant time so the time taken by the whole procedure is equal to the time taken by B plus an additional polynomial amount of work to compute $m(a)$.

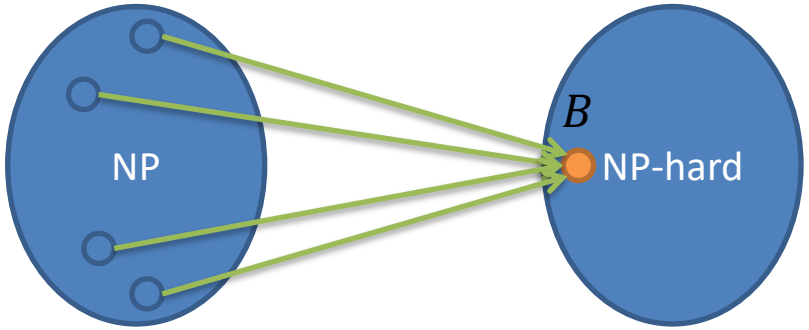


- Let's assume that we know that B is in P and that $A \leq_P B$. What does this tell us about A?
  - We now know that A is also in P.

NP-hard

- Consider a problem B for which every problem in NP reduces to B. That is

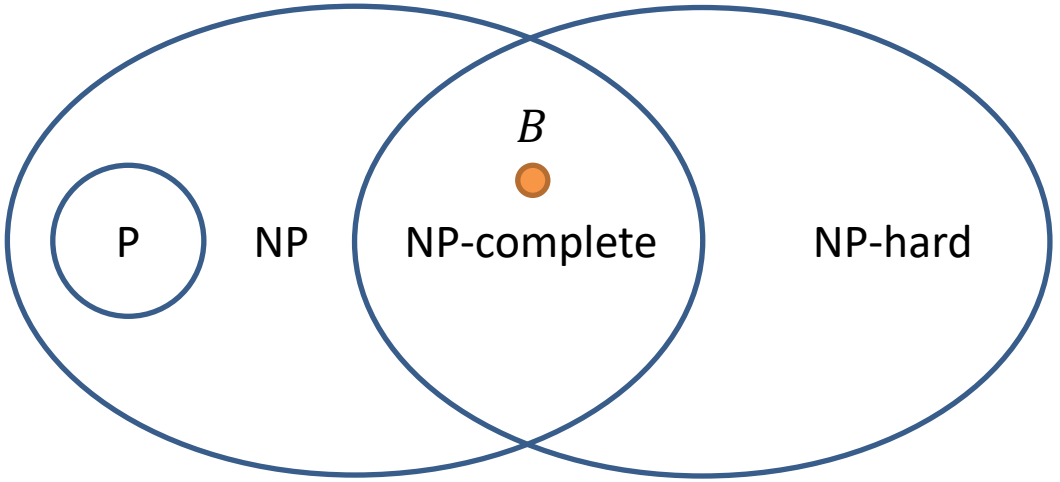
$$\forall_{A \in NP} : A \leq_P B$$



- Recall that $A \leq_P B$ means A is no harder than B.
  - So for this B every problem $A \in NP$ is no harder than B.
  - Another way of saying that is that B is harder than (or equal in complexity to) every problem in NP.
  - The class of problems that have this property are called NP-hard since they are the problems that are harder than every problem in NP.

NP-complete

- Imagine now a problem B that is in NP and is in NP-hard.
  - So B is harder than (or equal in complexity to) every problem in NP and it is in NP itself.
  - So B is an example of the hardest problem in NP.
  - We call these problems NP-complete problems.



P and NP-complete

Theorem: If $A \in P$ and $A \in NP$-complete then $P = NP$.

Proof: Assume $A \in P$ and $A \in NP$-complete.

Since $A \in P$ we know it can be solved by a deterministic algorithm in polynomial time.

Since $A \in NP$-complete we know that all problems in NP reduce to A in polynomial time.

Thus any problem in NP can be solved in polynomial time by computing the reduction to A and then solving A using the deterministic algorithm to solve A.

Thus every problem in NP is in P and $P = NP$. □

- So all we need to do is find a problem that is both in P and is NP-complete and we've shown $P = NP$.
  - We know of problems in P. We've seen examples already.
  - We also know of problems in NP-complete. We'll see some examples shortly.
  - However, we've never been able to find a problem in both P and NP-complete.

The First NP-complete Problem

Satisfiability (SAT)
Input: $b(x_1, x_2, \ldots, x_n)$ is a Boolean formula (in conjunctive normal form) of n variables
Output: "yes" if there exists an assignment of truth values to the variables that will make the formula true
"no" otherwise

- The first NP-complete problem was a version of SAT and was proven true by Stephen Cook in 1971.
  - Stephen_Cook
  - Independently the Soviet scientist Leonid Levin proved the theorem in 1969 and published it in 1973.
  - Leonid_Levin
  - Today the theorem is called the Cook-Levin theorem.
  - Cook-Levin_theorem
- In 1972 Richard Karp proved a list of 21 problems were NP-complete.
  - Richard_M._Karp
  - Karp's_21_NP-complete_problems
  - Cook and Karp won the Turing award in 1982 for this work (sorry Levin).
- The famous 1979 book by Garey and Johnson expanded this list to over a hundred NP-complete problems.
  - List_of_NP-complete_problems

The First NP-complete Problem

Satisfiability (SAT)
Input: $b(x_1, x_2, \ldots, x_n)$ is a Boolean formula (in conjunctive normal form) of n variables
Output: "yes" if there exists an assignment of truth values to the variables that will make the formula true
"no" otherwise

- How did Stephen Cook prove that SAT was NP-complete?
  - First to be NP-complete SAT must be in NP and SAT must be in NP-hard.
  - To show SAT is in NP we need to give a polynomial time verification procedure for SAT. Can we do this?
  - To show SAT is in NP-hard we need to reduce *every* problem in NP to SAT.

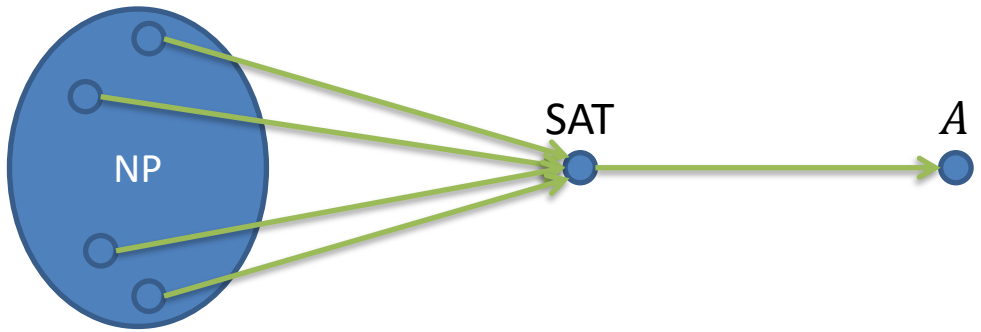
- Reducing every problem to SAT.
  - It's hard enough to reduce a single problem to SAT if you know something about the problem.
  - We don't even know all the (infinite) problems in NP so we need a very clever method to reduce all these problems to SAT.
  - The cleverness of Cook's and Levin's proofs is to encode the definition of a NP problem as a Boolean formula.
  - The details of Cook's proof is technical requiring some familiarity with Turing machines so we will not explore it here.

Transitivity of Reduction

- Consider problems A, B and C.

Theorem: If $A \leq_P B$ and $B \leq_P C$ then $A \leq_P C$.
Proof: If $A \leq_P B$ then there is a mapping $m(a)$ that maps inputs of A to inputs of B that preserves the output.
If $B \leq_P C$ then there is a mapping $p(b)$ that maps inputs of B to inputs of C that preserves the output.
By composing the two mappings we can create a mapping $p(m(a))$ that maps inputs of A to inputs of C that preserves the output.
Because $m(a)$ and $p(b)$ are polynomial time computable so is their composition. □

- How did Karp apply this idea?
  - Thanks to Cook (and Levin) there was now one known NP-complete problem SAT.
  - Since SAT is NP-complete every NP problem reduces to SAT.
  - If we can prove SAT reduces to another problem A then by transitivity every NP problem reduces to A.



Karp's 21 NP-complete Problems

- Once Cook (and Levin) did the hard work they opened a door for a second means of proving that a problem A is NP-complete.
  1. Select a known NP-complete problem B (like SAT).
  2. Show that $B \leq_P A$.
  3. Conclude that A is NP-complete.
- Since this method requires reducing a single known NP-complete problem to another problem not known to be NP-complete it is considerably easier than the initial proof.
  - This is why Karp was able to prove 21 problems soon after Cook's initial paper.
  - This is also why today there are hundreds of known NP-complete problems.
- Among Karp's initial 21 problems were a few we have seen already.
  - Subset Sum
  - Partition
  - Knapsack
  - Clique
  - Independent Set
  - Hamiltonian Cycle

Clique and Independent Set

Clique(CLIQUE)
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$.
Output: "yes" if there exists a complete subgraph of G of size k.
"no" otherwise

Independent Set(IS)
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$.
Output: "yes" if there exists a subgraph of G of size k with no edges.
"no" otherwise

- Let's show, as Karp did, that Clique reduces to Independent Set (and vice versa).
  - We have already shown that these problems are in NP. So if we show they reduce to one another and one of them is NP-complete then they both are.

- Reduction idea:
  - Consider a graph $G = (V, E)$.
  - Define $\bar{G} = (V, \bar{E})$ where $\bar{E} = \{e | e \notin E\}$ is the complement edge set.
  - Now if $V' \subseteq V$ is a clique in G then it is an independent set in $\bar{G}$.

- Notice that we can compute the complement graph in polynomial time.

Subset Sum, Partition, Knapsack

Subset Sum(SUM)
Input: $S[1 \dots n]$ a list of n positive integers and t a target integer.
Output: "yes" if there exists a subset of S that sums to t
"no" otherwise

Partition(PART)
Input: $S[1 \dots n]$ a list of n positive integers.
Output: "yes" if the set S can be partitioned into two sets such that each has the same sum
"no" otherwise

Knapsack(0-1 KNAP)
Input: $W[1 \dots n]$ a list of n positive weights $V[1 \dots n]$ a list of n positive values, c the capacity of the knapsack and t a threshold value
Output: "yes" if there exists $A \subseteq \{1, \dots, n\}$ a set of indexes of items such that $c \geq \sum_{i \in A} W[i]$ and $t \leq \sum_{i \in A} V[i]$
"no" otherwise

- These problems all reduce to one another. Let's see if we can prove these relationships:
  - Partition $\leq_P$ Subset Sum
  - Subset Sum $\leq_P$ Partition
  - Subset Sum $\leq_P$ Knapsack